

TELEGRAM FRAUD DETECTOR — FULL HANDOVER (FINAL)

1. PROJECT OVERVIEW

Project Name:

Telegram Fraud Detector (Multi-layer Phishing Detection System)

Objective:

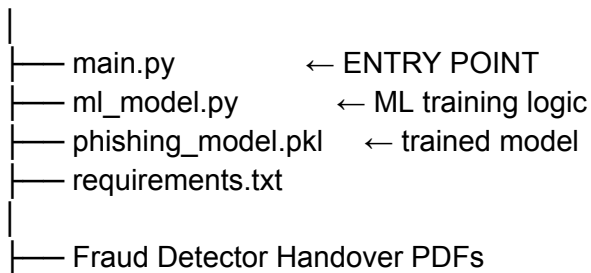
To detect **fraudulent / phishing messages and URLs** using:

- rule-based detection
 - threat intelligence datasets
 - API-based verification
 - machine learning
-

2. PROJECT STRUCTURE

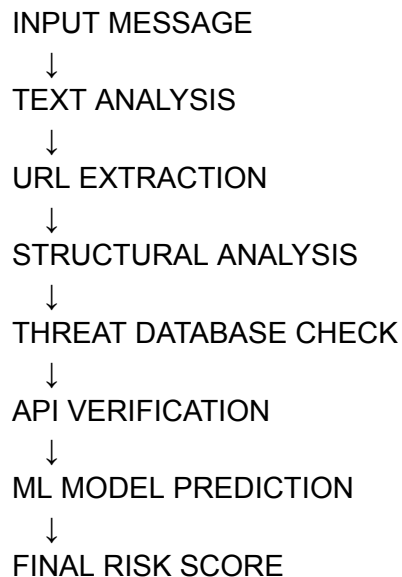
From your setup 📁

```
fraud-detector/
├── data/
│   ├── openphish.txt
│   ├── scam_patterns.txt
│   ├── threatfox.json
│   ├── url_dataset.csv
│   └── urlhaus.csv
├── detector/
│   ├── scam_detector.py ← CORE ENGINE
│   └── __pycache__/
```



3. SYSTEM ARCHITECTURE

Your system works in **multi-layer detection pipeline**:



4. DETECTION SIGNALS (COMPLETE LIST)

A. TEXT-LEVEL SIGNALS

- Suspicious keywords
- Scam pattern matching
- Jaccard similarity

- URL encoding detection
-

B. URL / DOMAIN SIGNALS

Structure-based

- URL present
 - Long URL
 - Too many dots
 - Too many hyphens
 - Too many numbers
 - Too many subdomains
-

Domain intelligence

- Domain not in popular list
 - Domain age (WHOIS API)
 - DNS failure
-

Advanced phishing detection

- Brand impersonation
 - Typosquatting (Levenshtein)
 - High entropy domain
 - Hidden URL mismatch
 - Short URL detection
 - Redirect detection
 - IP address detection
-

Newly strengthened signals

- Suspicious TLD (.xyz, .top)
 - HTTP vs HTTPS
 - Suspicious URL path
-



C. MACHINE LEARNING SIGNAL

From `ml_model.py`:

Features:

- URL length
- number of dots
- number of hyphens
- HTTPS presence
- IP presence

Model:

Random Forest Classifier



5. THREAT INTELLIGENCE SOURCES



OFFLINE DATASETS

Dataset	Purpose
OpenPhish	phishing URLs
URLhaus	malware URLs
ThreatFox	IOC intelligence
Scam patterns	message templates



ONLINE APIs

API	Function
Google Safe Browsing	malware/phishing detection
PhishTank	verified phishing

URLhaus API real-time malware

AbuseIPDB IP reputation

WHOIS API domain age

6. CORE FILES EXPLANATION

◆ **main.py**

entry point
takes user input
calls analyze_message()
prints score + reasons

◆ **scam_detector.py** (MOST IMPORTANT)

Contains:

- all detection logic
- feature extraction
- API integration
- scoring system

👉 This is your **core engine**

◆ **ml_model.py**

- trains ML model
 - saves **.pkl** file
-

◆ **phishing_model.pkl**

- trained model used in detection
-

7. SCORING SYSTEM

Range:

0 → 100

Score sources:

- rule-based signals
 - database matches
 - API results
 - ML prediction
-

Example:

High risk → phishing
Medium → suspicious
Low → safe

8. KEY FEATURES IMPLEMENTED

Multi-layer detection

- rule + ML + API

Real-time verification

- Safe Browsing

- AbuseIPDB

✓ Obfuscation detection

- encoding
- hidden URLs
- short links

✓ Advanced phishing detection

- typosquatting
 - entropy
 - subdomain abuse
-



9. LIMITATIONS (IMPORTANT)

- API rate limits
 - DNS failures may give false positives
 - ML model is basic (feature limited)
 - no caching (performance issue)
 - no UI / frontend
-



10. FUTURE IMPROVEMENTS



Immediate

- scoring balance
 - false positive reduction
 - caching APIs
-

Advanced

- Telegram bot integration
 - browser extension
 - real-time monitoring
 - deep learning model
-

11. CURRENT STATUS

Your project is now:

Advanced phishing detection system

Not beginner level anymore.

12. FINAL VERDICT

- ✓ Multi-layer detection
- ✓ Strong signal coverage
- ✓ Real-world datasets
- ✓ API integration
- ✓ ML support

👉 This is **very close to industry-level prototype**

NEXT STEP (IMPORTANT)

You already said:

👉 you want to:

- strengthen signals ✓ (almost done)

- then scoring system
- then reduce false positives